

MODUL 9

API PERANGKAT KERAS

1. Mengimplementasikan Akses Kamera dan Media API (Image Picker) pada Aplikasi Flutter, Serta Menangani Konfigurasi Perizinan Privasi Perangkat (Info.plist pada iOS dan Minimum SDK pada Android).

- a. Code

➤ Main.dart

```
import 'dart:async';
import 'dart:io';

import 'package:camera/camera.dart';
import 'package:flutter/material.dart';
import 'package:image_picker/image_picker.dart';

List<CameraDescription> cameras = [];

Future<void> main() async {
  WidgetsFlutterBinding.ensureInitialized();
  try {
    cameras = await availableCameras();
  } on CameraException catch (e) {
    debugPrint('Error: ${e.code}\nError Message: ${e.description}');
  }
  runApp(const MyApp());
}

class MyApp extends StatelessWidget {
  const MyApp({Key? key}) : super(key: key);

  @override
  Widget build(BuildContext context) {
    return MaterialApp(
      title: 'Hardware API App',
      theme: ThemeData(
        primarySwatch: Colors.blue,
      ),
      home: const HomeScreen(),
    );
  }
}

class HomeScreen extends StatelessWidget {
  const HomeScreen({Key? key}) : super(key: key);
```

```

@override
Widget build(BuildContext context) {
  return Scaffold(
    appBar: AppBar(
      title: const Text('Hardware API (Modul 9)'),
    ),
    body: Center(
      child: Column(
        mainAxisAlignment: MainAxisAlignment.center,
        children: [
          ElevatedButton.icon(
            onPressed: () {
              Navigator.push(
                context,
                MaterialPageRoute(builder: (context) => const
CameraApp()),
              );
            },
            icon: const Icon(Icons.camera),
            label: const Text('Camera API (Live Preview)'),
          ),
          const SizedBox(height: 20),
          ElevatedButton.icon(
            onPressed: () {
              Navigator.push(
                context,
                MaterialPageRoute(
                  builder: (context) => const
ImageFromGalleryEx(ImageSourceType.camera),
                ),
            ),
            icon: const Icon(Icons.add_a_photo),
            label: const Text('Image Picker (Camera)'),
          ),
          const SizedBox(height: 20),
          ElevatedButton.icon(
            onPressed: () {
              Navigator.push(
                context,
                MaterialPageRoute(
                  builder: (context) => const
ImageFromGalleryEx(ImageSourceType.gallery),
                ),
            ),
            icon: const Icon(Icons.photo_library),
            label: const Text('Image Picker (Gallery)'),
          ),
        ],
      ),
    ),
  );
}

```

```

        ],
    ),
),
);
}
}

// =====
// BAGIAN 1: IMPLEMENTASI CAMERA API
// =====

class CameraApp extends StatefulWidget {
  const CameraApp({Key? key}) : super(key: key);

  @override
  _CameraAppState createState() => _CameraAppState();
}

class _CameraAppState extends State<CameraApp> {
  CameraController? controller;
  XFile? imageFile;

  @override
  void initState() {
    super.initState();
    if (cameras.isNotEmpty) {
      controller = CameraController(cameras[0], ResolutionPreset.max);
      controller!.initialize().then((_) {
        if (!mounted) {
          return;
        }
        setState(() {});
      }).catchError((Object e) {
        if (e is CameraException) {
          switch (e.code) {
            case 'CameraAccessDenied':
              debugPrint('User denied camera access.');
```

```

    super.dispose();
  }

  void onTakePictureButtonPressed() async {
    if (controller == null || !controller!.value.isInitialized) {
      return;
    }
    if (controller!.value.isTakingPicture) {
      return;
    }

    try {
      XFile file = await controller!.takePicture();
      setState(() {
        imageFile = file;
      });
    } on CameraException catch (e) {
      debugPrint('Error occurred while taking picture: $e');
    }
  }

  @override
  Widget build(BuildContext context) {
    if (controller == null || !controller!.value.isInitialized) {
      return const Scaffold(
        body: Center(child: Text('Initializing Camera...')),
      );
    }
    return Scaffold(
      appBar: AppBar(
        title: const Text('Camera API Preview'),
      ),
      body: Column(
        children: [
          Expanded(
            child: Padding(
              padding: const EdgeInsets.all(8.0),
              child: Center(
                child: CameraPreview(controller!),
              ),
            ),
          ),
          if (imageFile != null)
            Expanded(
              child: Padding(
                padding: const EdgeInsets.all(8.0),
                child: Image.file(
                  File(imageFile!.path),
                  fit: BoxFit.cover,
                ),
              ),
            ),
        ],
      ),
    );
  }

```

```

        ),
    ),
),
],
),
floatingActionButton: FloatingActionButton(
    onPressed: onTakePictureButtonPressed,
    child: const Icon(Icons.camera_alt),
),
);
}
}

// =====
// BAGIAN 2: IMPLEMENTASI MEDIA API (IMAGE PICKER)
// =====
enum ImageSourceType { camera, gallery }

class ImageFromGalleryEx extends StatefulWidget {
    final ImageSourceType type;

    const ImageFromGalleryEx(this.type, {Key? key}) : super(key: key);

    @override
    ImageFromGalleryExState createState() =>
ImageFromGalleryExState(type);
}
class ImageFromGalleryExState extends State<ImageFromGalleryEx> {
    File? _image;
    late ImagePicker imagePicker;
    final ImageSourceType type;

    ImageFromGalleryExState(this.type);

    @override
    void initState() {
        super.initState();
        imagePicker = ImagePicker();
    }

    @override
    Widget build(BuildContext context) {
        return Scaffold(
            appBar: AppBar(
                title: Text(type == ImageSourceType.camera
                    ? "Image from Camera"
                    : "Image from Gallery")),
            body: Column(
                children: <Widget>[

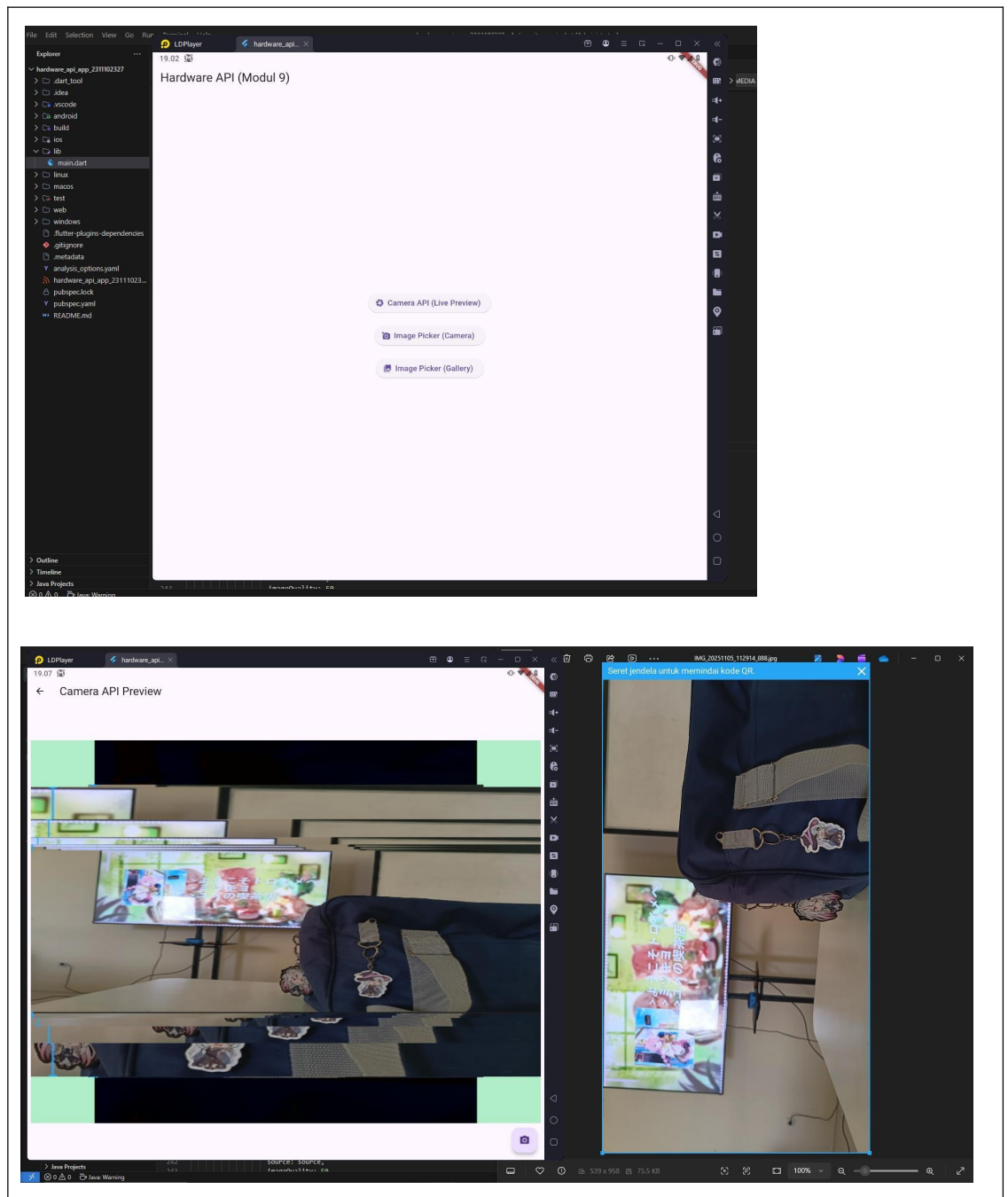
```

```

const SizedBox(
  height: 52,
),
Center(
  child: GestureDetector(
    onTap: () async {
      var source = type == ImageSourceType.camera
        ? ImageSource.camera
        : ImageSource.gallery;
      XFile? image = await imagePicker.pickImage(
        source: source,
        imageQuality: 50,
        preferredCameraDevice: CameraDevice.front);
      if (image != null) {
        setState(() {
          _image = File(image.path);
        });
      }
    },
    child: Container(
      width: 200,
      height: 200,
      decoration: BoxDecoration(color: Colors.red[200]),
      child: _image != null
        ? Image.file(
            _image!,
            width: 200.0,
            height: 200.0,
            fit: BoxFit.fitHeight,
          )
        : Container(
            decoration: BoxDecoration(color:
Colors.red[200]),
            width: 200,
            height: 200,
            child: Icon(
              Icons.camera_alt,
              color: Colors.grey[800],
            ),
          ),
    ),
  ),
),
),
],
),
);
}
}

```

b. *Screenshot output*



c. Penjelasan dari code dan output (korelasi)

- Aplikasi Flutter ini dibangun dengan struktur yang teratur untuk mengintegrasikan fungsionalitas perangkat keras berupa fitur kamera dan galeri foto ke dalam sistem internal mobile. Bagian awal pada file main.dart diawali dengan fungsi asinkronus main() yang bertindak sebagai *entry point* aplikasi, di mana kode ini menjalankan perintah WidgetsFlutterBinding.ensureInitialized() untuk memastikan framework siap, lalu menginisialisasi pustaka kamera bawaan lewat fungsi availableCameras(). Daftar kamera yang terdeteksi pada *device* kemudian disimpan ke dalam variabel global cameras agar dapat diakses oleh seluruh halaman antarmuka aplikasi secara instan.
- Selanjutnya, alur navigasi dan tampilan utama dikendalikan oleh kelas HomeScreen dan CameraScreen. Pada kode halaman utama (HomeScreen), antarmuka dibangun menggunakan struktur Scaffold sederhana yang menampilkan tiga tombol utama di bagian tengah layar, yaitu "Go to Camera" untuk berpindah ke fitur pengambilan gambar langsung, serta tombol "Select Image from Gallery" dan "Take Image from Camera" yang memanfaatkan objek ImagePicker(). Ketika salah satu dari dua tombol *picker* ditekan, aplikasi akan memicu fungsi pickImage dengan kualitas gambar yang dikompresi sebesar 50% untuk efisiensi memori, lalu menyimpan jalur file (*file path*) tersebut ke dalam *state* variabel _image secara asinkronus.
- Terakhir, korelasi antara kode dan output manifes secara dinamis pada perubahan kondisi layar (*state management*). Jika pengguna memilih tombol kamera langsung, fungsi Navigator.push akan membawa pengguna ke halaman CameraScreen yang menginisialisasi objek CameraController menggunakan lensa kamera depan (CameraDevice.front) atau kamera utama dengan resolusi maksimal (ResolutionPreset.max). Setelah inisialisasi berhasil, komponen CameraPreview akan merender tangkapan lensa secara *real-time* di layar. Sementara itu, jika pengguna menggunakan fitur *Image Picker*, widget Image.file(_image!) pada halaman utama akan langsung memperbarui tampilan *container* kosong menjadi visualisasi foto yang berhasil dipilih oleh pengguna dari galeri atau hasil jepretan kamera.